

5.1 Write a program to create a minimum spanning tree using Kruskal's algorithm. Write a program to create a minimum spanning tree using Prim "algorithm.

```
import java.util.*;
```

```
class Kruskal {
```

```
    class Edge implements Comparable<Edge> {
```

```
        int src, dest, weight;
```

```
        public int compareTo(Edge e) {
```

```
            return this.weight - e.weight;
```

```
        }
```

```
    }
```

```
    int V, E;
```

```
    Edge edge[];
```

```
    Kruskal(int v, int e) {
```

```
        V = v;
```

```
        E = e;
```

```
        edge = new Edge[E];
```

```
        for (int i = 0; i < e; i++)
```

```
            edge[i] = new Edge();
```

```
    }
```

```
    int find(int parent[], int i) {
```

```
        if (parent[i] == i)
```

```
            return i;
```

```
    return find(parent, parent[i]);  
}
```

```
void union(int parent[], int x, int y) {  
    int xset = find(parent, x);  
    int yset = find(parent, y);  
    parent[xset] = yset;  
}
```

```
void kruskalMST() {
```

```
    Edge result[] = new Edge[V];  
    for (int i = 0; i < V; i++)  
        result[i] = new Edge();
```

```
    Arrays.sort(edge);
```

```
    int parent[] = new int[V];  
    for (int i = 0; i < V; i++)  
        parent[i] = i;
```

```
    int e = 0;
```

```
    int i = 0;
```

```
    while (e < V - 1) {
```

```
        Edge next = edge[i++];
```

```
int x = find(parent, next.src);

int y = find(parent, next.dest);

if (x != y) {
    result[e++] = next;
    union(parent, x, y);
}
}

System.out.println("Edges in MST:");

for (i = 0; i < e; i++)
    System.out.println(result[i].src + " - " + result[i].dest + " : " + result[i].weight);
}

public static void main(String[] args) {

    int V = 4;
    int E = 5;

    Kruskal g = new Kruskal(V, E);

    g.edge[0].src = 0;
    g.edge[0].dest = 1;
    g.edge[0].weight = 10;

    g.edge[1].src = 0;
    g.edge[1].dest = 2;
```

```
g.edge[1].weight = 6;
```

```
g.edge[2].src = 0;
```

```
g.edge[2].dest = 3;
```

```
g.edge[2].weight = 5;
```

```
g.edge[3].src = 1;
```

```
g.edge[3].dest = 3;
```

```
g.edge[3].weight = 15;
```

```
g.edge[4].src = 2;
```

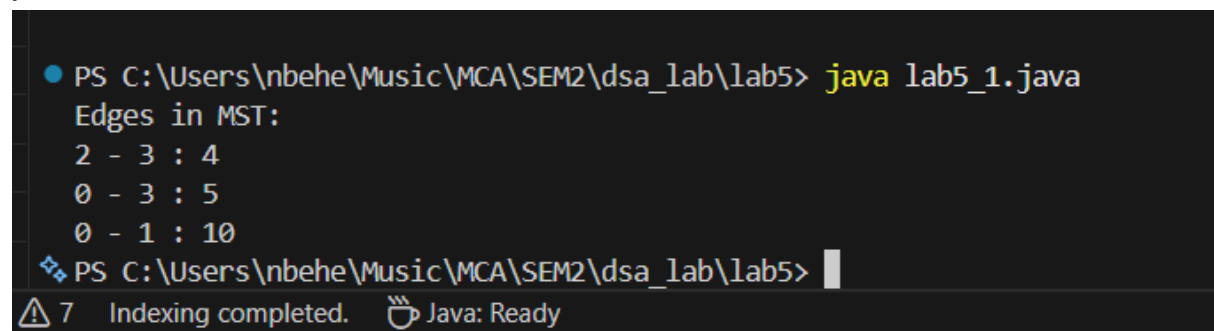
```
g.edge[4].dest = 3;
```

```
g.edge[4].weight = 4;
```

```
g.kruskalMST();
```

```
}
```

```
}
```



```
PS C:\Users\nbehe\Music\MCA\SEM2\dsa_lab\lab5> java lab5_1.java
Edges in MST:
2 - 3 : 4
0 - 3 : 5
0 - 1 : 10
PS C:\Users\nbehe\Music\MCA\SEM2\dsa_lab\lab5>
```

```
import java.util.*;
```

```
class Prim {
```

```
    static final int V = 5;
```

```
int minKey(int key[], boolean mstSet[]) {
    int min = Integer.MAX_VALUE, min_index = -1;

    for (int v = 0; v < V; v++)
        if (!mstSet[v] && key[v] < min) {
            min = key[v];
            min_index = v;
        }

    return min_index;
}

void printMST(int parent[], int graph[][] ) {
    System.out.println("Edge  Weight");
    for (int i = 1; i < V; i++)
        System.out.println(parent[i] + " - " + i + "  " + graph[i][parent[i]]);
}

void primMST(int graph[][] ) {

    int parent[] = new int[V];
    int key[] = new int[V];
    boolean mstSet[] = new boolean[V];

    for (int i = 0; i < V; i++) {
        key[i] = Integer.MAX_VALUE;
        mstSet[i] = false;
    }
}
```

```
key[0] = 0;
parent[0] = -1;

for (int count = 0; count < V - 1; count++) {

    int u = minKey(key, mstSet);
    mstSet[u] = true;

    for (int v = 0; v < V; v++)

        if (graph[u][v] != 0 && !mstSet[v] && graph[u][v] < key[v]) {
            parent[v] = u;
            key[v] = graph[u][v];
        }
    }

    printMST(parent, graph);
}

public static void main(String[] args) {

    Prim t = new Prim();

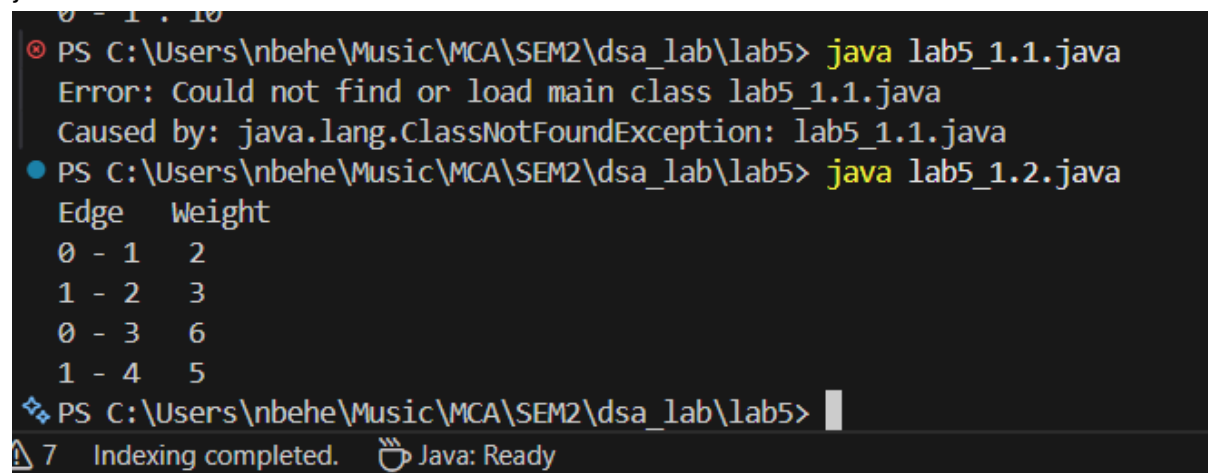
    int graph[][] = {
        {0, 2, 0, 6, 0},
        {2, 0, 3, 8, 5},
        {0, 3, 0, 0, 7},
```

```

        {6, 8, 0, 0, 9},
        {0, 5, 7, 9, 0},
    };

    t.primMST(graph);
}
}

```



```

PS C:\Users\nbehe\Music\MCA\SEM2\dsa_lab\lab5> java lab5_1.1.java
Error: Could not find or load main class lab5_1.1.java
Caused by: java.lang.ClassNotFoundException: lab5_1.1.java
PS C:\Users\nbehe\Music\MCA\SEM2\dsa_lab\lab5> java lab5_1.2.java
Edge  Weight
0 - 1   2
1 - 2   3
0 - 3   6
1 - 4   5
PS C:\Users\nbehe\Music\MCA\SEM2\dsa_lab\lab5>

```

5.2 Write a program to sort a given set of elements using Selection sort and hence find the time required to sort elements. Repeat the experiment for different values of n, the number of elements in the list to be sorted, and plot a graph of the time taken versus n.

```
import java.util.*;
```

```
class SelectionSortTime {
```

```
    static void selectionSort(int arr[]) {
```

```
        int n = arr.length;
```

```
        for (int i = 0; i < n - 1; i++) {
```

```
            int min = i;
```

```
        for (int j = i + 1; j < n; j++)
            if (arr[j] < arr[min])
                min = j;

        int temp = arr[min];
        arr[min] = arr[i];
        arr[i] = temp;
    }
}

public static void main(String args[]) {

    Scanner sc = new Scanner(System.in);

    System.out.print("Enter number of elements: ");
    int n = sc.nextInt();

    int arr[] = new int[n];
    Random r = new Random();

    for (int i = 0; i < n; i++)
        arr[i] = r.nextInt(1000);

    long start = System.nanoTime();

    selectionSort(arr);
```



```
long end = System.nanoTime();
```

```
System.out.println("Sorted elements:");
```

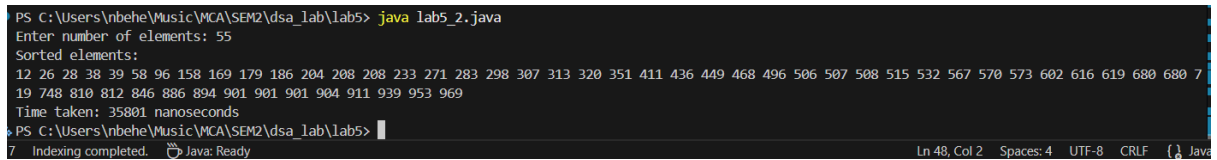
```
for (int i = 0; i < n; i++)
```

```
    System.out.print(arr[i] + " ");
```

```
System.out.println("\nTime taken: " + (end - start) + " nanoseconds");
```

```
}
```

```
}
```



```
PS C:\Users\nbehe\Music\MCA\SEM2\dsa_lab\lab5> java lab5_2.java
Enter number of elements: 55
Sorted elements:
12 26 28 38 39 58 96 158 169 179 186 204 208 208 233 271 283 298 307 313 320 351 411 436 449 468 496 506 507 508 515 532 567 570 573 602 616 619 680 680 7
19 748 810 812 846 886 894 901 901 901 904 911 939 953 969
Time taken: 35801 nanoseconds
PS C:\Users\nbehe\Music\MCA\SEM2\dsa_lab\lab5>
```

7 Indexing completed. Java: Ready Ln 48, Col 2 Spaces: 4 UTF-8 CRLF Java